CASE STUDY 14

Race Condition in xv6 Shared Data

A runnable C model of a free-list race and its spinlock fix

Operating Systems | Department of CSE

Chaitanya Bharathi Institute of Technology

Prepared by

Student	Roll number
L Nithish Kumar Goud	160124733327
G Ishan	160124733322
Santhosh	160124733332
Venkat	160124733340

27 October 2026

1. Case statement and scope

The assignment asks how concurrent access to shared kernel data can produce inconsistent updates, and how xv6 locking prevents them. This project studies the physical-page free list, kmem.freelist, and follows one synchronization method: a spinlock. [1]

The deliverable includes a runnable host-side C model and a source-level comparison with xv6. The C program forces a specific two-thread interleaving, observes the duplicate allocation, then repeats the pop operation with a spinlock. It does not boot xv6 or change the xv6 kernel source.

Objectives covered

- Explain the shared free-list data and the invariant it must preserve.
- Show a critical section where each allocation reads and changes the same head pointer.
- Reproduce a duplicate allocation in a controlled unlocked run.
- Protect the complete list pop with one spinlock and compare the resulting state.

2. Shared data and its invariant

In xv6, kmem.freelist points to the first free physical page. Each page stores a link to the next page. kalloc() removes the first page; kfree() inserts a page at the head. Both operations use kmem.lock around the list changes in the local xv6 source. [2]

State	Required condition
Free page	Appears exactly once in the reachable free list.
Allocated page	Is returned to one caller at a time.
Whole allocator state	Each page belongs to either the free list or one allocation.

Table 1. The allocator invariant checked by the C demonstration.

Relevant xv6 kalloc() operation

```
acquire(&kmem.lock);
r = kmem.freelist;
if (r)
  kmem.freelist = r->next;
release(&kmem.lock);
```

The lock must cover both the read and the head update. Protecting only one pointer access would still let a second CPU act on a stale head.

3. Inconsistent update without the lock

Assume the list begins P0 -> P1 -> P2. If two CPU threads read the head before either advances it, both hold P0 as their candidate. Each then stores P1 as the new head and returns P0. The list head is advanced, but two callers now believe they own the same physical page.

Step	CPU 0	CPU 1	Shared state
1	Read head: P0	-	P0 -> P1 -> P2
2	-	Read head: P0	P0 -> P1 -> P2

Step	CPU 0	CPU 1	Shared state
3	Store head: P1	-	P1 -> P2
4	-	Store head: P1	P1 -> P2
5	Return P0	Return P0	P0 has two owners

Table 2. A possible source-level interleaving. It is the schedule forced by the host demo, not an xv6 hardware trace.

Possible impact: duplicate page ownership can expose two processes to the same physical memory, while other lost or damaged list updates can make a free page unreachable. The C demo specifically reproduces duplicate allocation; it does not claim to reproduce every allocator failure.

4. One method: xv6-style spinlock

The spinlock gives one CPU exclusive access to the free-list critical section. The first CPU acquires the lock, reads P0, advances the head to P1, then releases the lock. The second CPU acquires the lock afterward and reads P1. The lock therefore protects the invariant across the complete read-modify-write operation. [2, 3]

Phase	Protected action
Acquire	Claim kmem.lock with atomic ownership and acquire ordering.
Critical section	Read the head and link the removed page out of the list.
Release	Publish the completed update before another CPU enters.

Table 3. The spinlock sequence used by xv6 and mirrored in the program.

The local xv6 spinlock implementation uses an atomic exchange to acquire ownership and a release store to unlock. It also disables local interrupts while holding a spinlock to avoid a same-CPU interrupt deadlock. The host program mirrors the acquire/release lock idea with GCC atomic built-ins and pthread worker threads. [3]

5. C implementation

The complete program is saved as src/race_demo.c. It creates a three-page list and two worker threads. The unlocked test uses a two-party pthread barrier as test instrumentation so both workers read P0 before either updates the head. The barrier fixes the interleaving for demonstration; it is not the repair.

The host demo performs atomic loads and stores on the head pointer even in the unlocked case. This avoids relying on a C data race to illustrate the allocator bug. Each individual pointer access is atomic, but the whole pop is not. The spinlock is the single method that makes the combined operation exclusive.

Build and run

```
cd "/mnt/c/Users/HP/Documents/OS record/OS CASE STUDY/case_study_14_rebuild"
make clean
make run
```

Environment for the recorded run: WSL Ubuntu 24.04, GCC 13.3.0. The Makefile compiles src/race_demo.c with -std=c11 -Wall -Wextra -Wpedantic -O2 -pthread.

6. Recorded result

```
make: Entering directory '/mnt/c/Users/HP/Documents/OS record/OS CASE
STUDY/case_study_14_rebuild'
./build/race_demo
XV6-style physical-page free-list demonstration
Two CPU threads, three pages, one allocation per thread

UNLOCKED RUN (controlled interleaving)
  CPU 0 read head P0; CPU 1 read head P0
  Allocations: CPU 0 = P0, CPU 1 = P0
  Same page returned twice: YES
  Free-list head after both pops: P1
  Pages present exactly once (allocated + free): 2 / 3
  State: INCONSISTENT

SPINLOCK RUN (lock covers the complete pop)
  Allocations: CPU 0 = P0, CPU 1 = P1
  Same page returned twice: NO
  Free-list head after both pops: P2
  Pages present exactly once (allocated + free): 3 / 3
  State: CONSISTENT

DEMONSTRATION CHECK: PASS (race reproduced; spinlock preserved the invariant)
make: Leaving directory '/mnt/c/Users/HP/Documents/OS record/OS CASE STUDY/case_study_14_rebuild'
```

Figure 1. Console output captured from the clean project run in evidence/run_output.txt.

Run mode	Allocations	Pages exactly once	Observed state
Unlocked	CPU 0 = P0, CPU 1 = P0	2 / 3	INCONSISTENT
Spinlock	CPU 0 = P0, CPU 1 = P1	3 / 3	CONSISTENT

Table 4. The actual run reproduces the duplicate page and then passes the invariant check with the spinlock.

The demonstration check reports PASS because the expected contrast occurred: the unlocked schedule returned one page twice, while the spinlock run accounted for all three pages exactly once across allocations and the remaining list.

7. Conclusion and limits

The experiment shows why a shared allocator needs mutual exclusion across the whole operation that updates its free-list head. Two stale reads can return the same page. With the spinlock held across the read and update, the second caller observes the new head and receives another page.

This controlled host-side example demonstrates one harmful interleaving and its spinlock fix. It is not a kernel experiment: it does not boot xv6, pin threads to physical CPU cores, measure lock contention, or explore all schedules. The result supports the stated invariant for this scenario and should not be read as a claim about every possible xv6 run.

References

[1] Case Study 14 assignment brief, “Race Condition in XV6 Shared Data,” supplied with the project.

[2] MIT PDOS xv6-riscv source, kernel/kalloc.c, local revision ae7cff64d1cd1153e98794d19c46235ead934cf2.

[3] MIT PDOS xv6-riscv source, kernel/spinlock.c, local revision ae7cff64d1cd1153e98794d19c46235ead934cf2.

Appendix A. Complete C source

The listing below is the same source that produced the recorded output. The runnable file is also included at src/race_demo.c.

```
#define _XOPEN_SOURCE 700
#include <pthread.h>
#include <sched.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* A small user-space model of xv6's kernel/kalloc.c free list. */
enum { NCPU = 2, NPAGES = 3 };

struct run {
    int id;
    struct run *next;
};

struct spinlock {
    int locked;
};

static struct run pages[NPAGES];
static struct run *freelist;
static struct spinlock kmem_lock;
static pthread_barrier_t both_cpus_loaded;
static struct run *observed[NCPU];
static struct run *allocated[NCPU];

struct worker_arg {
    int cpu;
};

/* Same spinlock idea used by xv6: atomic ownership with acquire/release order. */
static void acquire(struct spinlock *lock)
{
    while (__atomic_exchange_n(&lock->locked, 1, __ATOMIC_ACQUIRE) != 0)
        sched_yield();
}

static void release(struct spinlock *lock)
{
    __atomic_store_n(&lock->locked, 0, __ATOMIC_RELEASE);
}

static void init_free_list(void)
{
    for (int i = 0; i < NPAGES; ++i) {
        pages[i].id = i;
        pages[i].next = (i + 1 < NPAGES) ? &pages[i + 1] : NULL;
    }

    __atomic_store_n(&freelist, &pages[0], __ATOMIC_RELAXED);
    __atomic_store_n(&kmem_lock.locked, 0, __ATOMIC_RELAXED);
    memset(observed, 0, sizeof(observed));
    memset(allocated, 0, sizeof(allocated));
}

/*
 * Deliberately unsafe pop. The test barrier forces both CPUs to read the same
 * head before either publishes its new head. Atomic pointer operations keep
 * this host-side demonstration defined, but they do not protect the complete
 * read-modify-write operation.
 */
static void *unsafe_worker(void *arg)
{
    const int cpu = ((struct worker_arg *)arg)->cpu;
    struct run *page = __atomic_load_n(&freelist, __ATOMIC_RELAXED);
    struct run *next = page ? page->next : NULL;
    observed[cpu] = page;

    const int barrier_status = pthread_barrier_wait(&both_cpus_loaded);
    if (barrier_status != 0 && barrier_status != PTHREAD_BARRIER_SERIAL_THREAD) {
        fprintf(stderr, "pthread_barrier_wait failed: %s\n", strerror(barrier_status));
        abort();
    }

    if (page != NULL)
        __atomic_store_n(&freelist, next, __ATOMIC_RELAXED);
    allocated[cpu] = page;
    return NULL;
}
```

```
/* The spinlock covers both the head read and the head update as one operation. */
static void *spinlock_worker(void *arg)
{
    const int cpu = ((struct worker_arg *)arg)->cpu;

    acquire(&kmem_lock);
    struct run *page = __atomic_load_n(&freelist, __ATOMIC_RELAXED);
    if (page != NULL)
        __atomic_store_n(&freelist, page->next, __ATOMIC_RELAXED);
    release(&kmem_lock);

    allocated[cpu] = page;
    return NULL;
}

static int run_pair(void *(*worker)(void *), struct worker_arg args[NCPU])
{
    pthread_t threads[NCPU];

    for (int i = 0; i < NCPU; ++i) {
        const int rc = pthread_create(&threads[i], NULL, worker, &args[i]);
        if (rc != 0) {
            fprintf(stderr, "pthread_create failed: %s\n", strerror(rc));
            return -1;
        }
    }

    for (int i = 0; i < NCPU; ++i) {
        const int rc = pthread_join(threads[i], NULL);
        if (rc != 0) {
            fprintf(stderr, "pthread_join failed: %s\n", strerror(rc));
            return -1;
        }
    }
    return 0;
}

static int page_id(const struct run *page)
{
    return page ? page->id : -1;
}

/* Verify every page appears exactly once across allocations and the free list. */
static bool audit_state(int *exactly_once)
{
    int occurrences[NPAGES] = {0};
    bool seen_in_list[NPAGES] = {false};
    bool valid = true;
    int steps = 0;

    for (int cpu = 0; cpu < NCPU; ++cpu) {
        if (allocated[cpu] != NULL) {
            const int id = page_id(allocated[cpu]);
            if (id < 0 || id >= NPAGES) {
                valid = false;
            } else {
                ++occurrences[id];
            }
        }
    }

    struct run *page = __atomic_load_n(&freelist, __ATOMIC_RELAXED);
    while (page != NULL && steps < NPAGES) {
        const int id = page_id(page);
        if (id < 0 || id >= NPAGES || seen_in_list[id]) {
            valid = false;
            break;
        }
        seen_in_list[id] = true;
        ++occurrences[id];
        ++steps;
        page = page->next;
    }
    if (page != NULL)
        valid = false;

    *exactly_once = 0;
    for (int i = 0; i < NPAGES; ++i) {
        if (occurrences[i] == 1)
            ++*exactly_once;
        else
            valid = false;
    }
    return valid;
}

static void print_state(const char *label, bool show_observed)
{
    int exactly_once = 0;
```

```
    const bool consistent = audit_state(&exactly_once);
    const struct run *head = __atomic_load_n(&freelist, __ATOMIC_RELAXED);

    printf("%s\n", label);
    if (show_observed) {
        printf("  CPU 0 read head P%d; CPU 1 read head P%d\n",
               page_id(observed[0]), page_id(observed[1]));
    }
    printf("  Allocations: CPU 0 = P%d, CPU 1 = P%d\n",
           page_id(allocated[0]), page_id(allocated[1]));
    printf("  Same page returned twice: %s\n",
           allocated[0] != NULL && allocated[0] == allocated[1] ? "YES" : "NO");
    printf("  Free-list head after both pops: P%d\n", page_id(head));
    printf("  Pages present exactly once (allocated + free): %d / %d\n",
           exactly_once, NPAGES);
    printf("  State: %s\n\n", consistent ? "CONSISTENT" : "INCONSISTENT");
}

int main(void)
{
    struct worker_arg args[NCPU] = {{.cpu = 0}, {.cpu = 1}};
    const int barrier_rc = pthread_barrier_init(&both_cpus_loaded, NULL, NCPU);
    if (barrier_rc != 0) {
        fprintf(stderr, "pthread_barrier_init failed: %s\n", strerror(barrier_rc));
        return 1;
    }

    printf("XV6-style physical-page free-list demonstration\n");
    printf("Two CPU threads, three pages, one allocation per thread\n\n");

    init_free_list();
    if (run_pair(unsafe_worker, args) != 0)
        return 1;
    print_state("UNLOCKED RUN (controlled interleaving)", true);
    const bool unsafe_reproduced = allocated[0] != NULL &&
                                   allocated[0] == allocated[1] &&
                                   page_id(__atomic_load_n(&freelist, __ATOMIC_RELAXED)) == 1;

    const int destroy_rc = pthread_barrier_destroy(&both_cpus_loaded);
    if (destroy_rc != 0) {
        fprintf(stderr, "pthread_barrier_destroy failed: %s\n", strerror(destroy_rc));
        return 1;
    }

    init_free_list();
    if (run_pair(spinlock_worker, args) != 0)
        return 1;
    print_state("SPINLOCK RUN (lock covers the complete pop)", false);
    int exact_count = 0;
    const bool spinlock_consistent = audit_state(&exact_count) && exact_count == NPAGES;
    const bool demonstration_passed = unsafe_reproduced && spinlock_consistent;

    printf("DEMONSTRATION CHECK: %s\n",
           demonstration_passed
               ? "PASS (race reproduced; spinlock preserved the invariant)"
               : "FAIL (expected outcomes were not observed)");
    return demonstration_passed ? 0 : 1;
}
```